

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

2026.06.08

C++ Class 개념정리

핵심

클래스는 데이터와 함수를 하나로 묶어 객체를 만들기 위한 설계도이다. C++에서 클래스는 캡슐화, 생성자, 소멸자, 상속, 오버로딩, 오버라이딩, 가상함수, static 같은 객체지향 개념을 이해하는 중심이다.

Class

클래스는 객체를 만들기 위한 설계도이다. 예를 들어 Book 클래스는 책 객체를 만들기 위한 설계도이고, 실제로 만들어진 book1, book2 같은 값이 객체이다.

```
class Book {  
private:  
    string title;  
    string author;  
  
public:  
    void setTitle(string t) {  
        title = t;  
    }  
  
    string getTitle() {  
        return title;  
    }  
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`title`, `author`는 멤버 변수이고, `setTitle()`, `getTitle()`은 멤버 함수이다. 멤버 변수는 객체가 가지는 값이고, 멤버 함수는 객체가 수행하는 동작이다.

접근 지정자

접근 지정자는 클래스 안의 멤버를 어디에서 사용할 수 있는지 정한다.

public

클래스 밖에서도 접근할 수 있다.

private

클래스 안에서만 접근할 수 있다. 보통 멤버 변수는 `private`으로 숨긴다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

protected

상속 관계에서 자식 클래스가 접근할 수 있다.

정리 멤버 변수를 private으로 숨기고, public 함수로 값을 설정하거나 가져오게 하면 데이터를 더 안전하게 관리할 수 있다. 이것이 캡슐화의 기본 형태이다.

다중 클래스

다중 클래스는 하나의 프로그램 안에서 여러 클래스를 역할별로 나누어 사용하는 구조이다.

```
class Book {
public:
    string title;
    string author;
};

class Library {
private:
    Book books[100];
    int count;

public:
    void addBook(Book book) {
        books[count] = book;
        count++;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 'Book' 클래스는 책 한 권의 정보를 담당하고, 'Library' 클래스는 여러 책을 관리한다. 이렇게 나누면 한 클래스가 너무 커지는 것을 막고 역할이 분명해진다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

생성자

생성자는 객체가 만들어질 때 자동으로 실행되는 함수이다. 객체의 초기값을 설정할 때 많이 사용한다.

```
class Book {
private:
string title;

public:
Book(string t) {
title = t;
}
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
Book book("C Programming");
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

생성자는 클래스 이름과 같고 반환형이 없다. 객체가 생성될 때 자동으로 실행된다.

생성자 오버로딩

생성자는 여러 개 만들 수 있다. 매개변수의 개수나 자료형이 다르면 같은 이름의 생성자를 여러 개 둘 수 있다.

```
class Book {
public:
Book() {
cout << "기본 생성자" << endl;
}

Book(string title) {
cout << title << endl;
}
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

소멸자

소멸자는 객체가 사라질 때 자동으로 실행되는 함수이다.

```
class Book {
public:
    ~Book() {
        cout << "객체 삭제" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

소멸자는 클래스 이름 앞에 `~`를 붙인다. 반환형과 매개변수가 없다. 동적 메모리 해제, 파일 닫기 같은 정리 작업에 사용한다.

상속

상속은 부모 클래스의 기능을 자식 클래스가 물려받는 것이다.

```
class Person {
public:
    string name;
};

class Student : public Person {
public:
    int score;
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`Student`는 `Person`의 `name`을 물려받고, 추가로 `score`를 가진다. 상속을 사용하면 공통 기능을 재사용할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

다중 상속

다중 상속은 하나의 클래스가 여러 부모 클래스를 동시에 상속받는 것이다.

```
class Printer {
public:
    void print() {
        cout << "출력" << endl;
    }
};

class Scanner {
public:
    void scan() {
        cout << "스캔" << endl;
    }
};

class MultiFunction : public Printer, public Scanner {
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

다중 상속은 여러 기능을 조합할 수 있다는 장점이 있지만, 부모 클래스에 같은 이름의 함수나 변수가 있으면 충돌할 수 있고 구조가 복잡해질 수 있다.

오버로딩

오버로딩은 같은 이름의 함수를 여러 개 만드는 것이다. 단, 매개변수의 개수나 자료형이 달라야 한다.

```
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

오버로딩은 같은 기능을 하는 함수에 같은 이름을 사용할 수 있게 해준다. 생성자 오버로딩도 가능하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

오버라이딩

오버라이딩은 부모 클래스의 함수를 자식 클래스에서 다시 정의하는 것이다.

```
class Animal {
public:
virtual void sound() {
cout << "동물 소리" << endl;
}
};

class Dog : public Animal {
public:
void sound() override {
cout << "멍멍" << endl;
}
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

오버라이딩은 상속 관계에서 발생한다. 부모 함수와 자식 함수의 이름과 매개변수 형태가 같아야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

오버라이딩을 어디에 쓰는가

오버라이딩은 같은 명령을 내려도 객체 종류에 따라 다르게 행동해야 할 때 사용한다.

게임 캐릭터 예시: 게임에 `Character`라는 부모 클래스가 있고, `Warrior`, `Mage`, `Archer`라는 자식 클래스가 있다고 해보자. 모두 공격할 수 있지만 공격 방식은 다르다.

```
class Character {
public:
    virtual void attack() {
        cout << "기본 공격" << endl;
    }
};

class Warrior : public Character {
public:
    void attack() override {
        cout << "검으로 공격" << endl;
    }
};

class Mage : public Character {
public:
    void attack() override {
        cout << "마법으로 공격" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `attack()`이라는 이름은 같지만 전사는 검, 마법사는 마법으로 공격한다. 이런 상황에서 오버라이딩을 사용한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

가상함수

가상함수는 부모 클래스의 포인터나 참조로 자식 객체를 다룰 때, 실제 자식 클래스의 함수가 실행되도록 해주는 함수이다.

```
class Animal {
public:
virtual void sound() {
cout << "동물 소리" << endl;
}
};

class Dog : public Animal {
public:
void sound() override {
cout << "멍멍" << endl;
}
};

int main() {
Animal* animal = new Dog();
animal->sound();
delete animal;
return 0;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 `sound()`가 가상함수이면 부모 포인터로 호출해도 실제 객체인 `Dog`의 `sound()`가 실행된다.

핵심 가상함수는 다형성을 구현하기 위해 필요하다. 같은 `sound()` 호출이라도 실제 객체가 Dog인지 Cat인지에 따라 다르게 실행되도록 만든다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

가상함수를 어디에 쓰는가

가상함수는 부모 타입 하나로 여러 자식 객체를 관리할 때 사용한다.

게임 캐릭터 예시:

```
Character* party[2];
party[0] = new Warrior();
party[1] = new Mage();

party[0]->attack();
party[1]->attack();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

```
검으로 공격
마법으로 공격
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가: 배열의 타입은 모두 `Character\*`이지만 실제 객체는 `Warrior`, `Mage`이다. 이때 `attack()`이 가상함수이면 실제 캐릭터 종류에 맞는 공격이 실행된다.

비유

게임에서 플레이어가 "공격!" 버튼을 누르면 캐릭터 종류에 따라 행동이 달라진다. 전사는 검을 휘두르고, 마법사는 마법을 쓰고, 궁수는 활을 쏜다. 버튼 이름은 하나지만 실제 동작은 캐릭터마다 다르다. 이것이 가상함수와 오버라이딩이 필요한 이유이다.

순수 가상함수

순수 가상함수는 부모 클래스에서 함수 형태만 정하고, 실제 구현은 자식 클래스가 반드시 하도록 만드는 함수이다.

```
class Animal {
public:
    virtual void sound() = 0;
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

`= 0`이 붙은 함수가 순수 가상함수이다. 순수 가상함수를 가진 클래스는 추상 클래스이고, 직접 객체를 만들 수 없다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

static

`static` 멤버는 객체마다 따로 생기는 값이 아니라 클래스 전체가 공유하는 값이다.

```
class Student {
public:
    static int count;

    Student() {
        count++;
    }
};

int Student::count = 0;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
Student s1;
Student s2;
cout << Student::count << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 `count`는 객체마다 따로 존재하는 값이 아니라 `Student` 클래스 전체가 공유하는 값이다.

static을 쓰는 이유

`static`을 쓰는 가장 큰 이유는 객체마다 따로 관리할 필요가 없는 값이나 기능을 클래스 전체에서 공유하기 위해서이다.

객체 개수 세기

학생 객체가 몇 개 만들어졌는지 세려면 모든 객체가 같은 카운터를 공유해야 한다.

공통 설정값 저장

모든 객체가 같은 설정값을 사용해야 할 때 static 변수를 사용할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

객체 없이 함수 사용

단순 계산 함수처럼 객체의 상태를 사용하지 않는 함수는 static 함수로 만들 수 있다.

```
class Math {  
public:  
    static int add(int a, int b) {  
        return a + b;  
    }  
};  
  
cout << Math::add(10, 20) << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리 `static`은 객체 개별 정보가 아니라 클래스 공통 정보나 공통 기능을 표현할 때 사용한다.

static을 어디에 쓰는가

게임에서 현재 생성된 몬스터 수를 세거나, 전체 플레이어 수를 관리할 때 사용할 수 있다.

```
class Monster {  
public:  
    static int monsterCount;  
  
    Monster() {  
        monsterCount++;  
    }  
};  
  
int Monster::monsterCount = 0;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
Monster m1;  
Monster m2;  
  
cout << Monster::monsterCount << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

2

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 몬스터 수는 각 몬스터가 따로 가지는 값이 아니라 게임 전체가 공유해야 하는 값이다. 그래서 `static`이 적합하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

게임으로 이해하는 Class 개념

Class 개념은 게임 캐릭터로 비유하면 이해하기 쉽다. 부모 클래스는 모든 챔피언의 공통 설계도이고, 자식 클래스는 실제 챔피언이라고 생각하면 된다.

전체 비유:

```
부모 클래스 = Champion
자식 클래스 = Garen, Ahri, Ezreal
공통 기능 = move(), attack(), useSkill()
각 챔피언만의 기능 = 스킬 효과, 공격 방식
static 값 = 전체 챔피언 수
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

부모 클래스 Champion

모든 챔피언은 이름, 체력, 이동, 공격, 스킬 사용 같은 공통 특징을 가진다. 그래서 이 공통 부분을 부모 클래스 `Champion`에 만든다.

```
class Champion {
public:
    string name;
    int hp;

    void move() {
        cout << "이동한다" << endl;
    }

    virtual void useSkill() {
        cout << "기본 스킬 사용" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`Champion`은 모든 챔피언의 기본 설계도이다. `move()`처럼 모든 챔피언이 공통으로 하는 행동은 부모 클래스에 넣는다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

상속

상속은 부모 클래스의 기능을 자식 클래스가 물려받는 것이다.

```
class Garen : public Champion {
};

class Ahri : public Champion {
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

설명: `Garen`과 `Ahri`는 `Champion`을 상속받았으므로 `name`, `hp`, `move()` 같은 기본 기능을 물려받는다.

왜 필요한가: 모든 챔피언마다 `name`, `hp`, `move()`를 다시 만들면 코드가 반복된다. 공통 기능은 부모 클래스에 한번만 작성하고, 자식 클래스는 다른 부분만 추가하면 된다.

오버라이딩

오버라이딩은 부모 클래스의 함수를 자식 클래스에서 자기 방식으로 다시 정의하는 것이다.

```
class Garen : public Champion {
public:
void useSkill() override {
cout << "가렌: 데마시아의 정의!" << endl;
}
};

class Ahri : public Champion {
public:
void useSkill() override {
cout << "아리: 매혹 사용!" << endl;
}
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

설명: 함수 이름은 둘 다 `useSkill()`이지만, 가렌은 가렌 스킬을 쓰고 아리는 아리 스킬을 쓴다.

왜 필요한가: "스킬 사용"이라는 명령은 같지만 실제 스킬 효과는 챔피언마다 다르기 때문이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

가상함수 virtual

가상함수는 부모 타입으로 자식 객체를 다룰 때 실제 자식 클래스의 함수가 실행되게 해준다.

```

Champion* champion = new Garen();
champion->useSkill();

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

가렌: 데마시아의 정의!

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

설명: 변수 타입은 `Champion\*`이지만 실제 객체는 `Garen`이다. `useSkill()`이 `virtual`이면 부모 함수가 아니라 실제 객체인 `Garen`의 함수가 실행된다.

게임 팀 배열 예시:

```

Champion* team[3];
team[0] = new Garen();
team[1] = new Ahri();
team[2] = new Ezreal();

team[0]->useSkill();
team[1]->useSkill();
team[2]->useSkill();

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가: 팀 목록은 전부 `Champion\*`로 관리하지만, 스킬을 사용할 때는 실제 챔피언에 맞는 스킬이 나가야 한다. 이것이 가상함수와 다형성이 필요한 이유이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

오버로딩

오버로딩은 같은 이름의 함수를 매개변수를 다르게 해서 여러 개 만드는 것이다.

```
class Champion {
public:
    void attack() {
        cout << "기본 공격" << endl;
    }

    void attack(string target) {
        cout << target << "을 공격" << endl;
    }

    void attack(string target, int damage) {
        cout << target << "에게 " << damage << " 피해" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

사용 예:

```
champion.attack();
champion.attack("미니언");
champion.attack("적 챔피언", 100);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가: 공격이라는 의미는 같지만, 어떤 상황에서는 대상이 필요하고 어떤 상황에서는 데미지도 필요하다. 함수 이름을 `attackBasic`, `attackTarget`, `attackDamage`처럼 나누는 것보다 `attack()`으로 통일하는 것이 깔끔하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

다중 상속

다중 상속은 하나의 클래스가 여러 부모 클래스를 동시에 상속받는 것이다.

```
class AttackSkill {
public:
    void attackSkill() {
        cout << "공격 스킬 사용" << endl;
    }
};

class HealSkill {
public:
    void healSkill() {
        cout << "회복 스킬 사용" << endl;
    }
};

class Paladin : public AttackSkill, public HealSkill {
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

설명: `Paladin`은 공격 스킬도 쓰고 회복 스킬도 쓸 수 있는 챔피언이다.

왜 필요한가: 여러 기능을 조합하고 싶을 때 사용할 수 있다. 하지만 부모 클래스들에 같은 이름의 함수가 있으면 충돌할 수 있으므로 조심해야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

static

`static`은 객체마다 따로 생기는 값이 아니라 클래스 전체가 공유하는 값이다.

게임에서는 전체 챔피언 수나 전체 몬스터 수처럼 모든 객체가 함께 봐야 하는 값을 `static`으로 만들 수 있다.

```
class Champion {
public:
    static int championCount;

    Champion() {
        championCount++;
    }
};
```

```
int Champion::championCount = 0;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

사용 예:

```
Champion c1;
Champion c2;
Champion c3;

cout << Champion::championCount << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과

결과

3

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

설명: 체력은 챔피언마다 따로 있어야 하지만, 전체 챔피언 수는 모든 챔피언이 공유해야 한다. 그래서 `static`을 사용한다.

게임 비유 최종표

| 개념 | 게임 비유 | 필요한 이유 |

|---|---|---|

| 클래스 | Champion 설계도 | 챔피언 객체를 만들기 위해 || 객체 | Garen, Ahri 같은 실제 챔피언 | 실제 게임 안에서 동작하는 대상 || 상속 | Garen이 Champion의 기본 기능을 물려받음 | 공통 기능 중복 제거 || 오버라이딩 | 챔피언마다 `useSkill()`을 다르게 구현 | 같은 명령, 다른 행동 || 가상함수 | Champion 포인터로 불러도 Garen 스킬이 나감 | 실제 객체 기준 실행 || 오버로딩 | `attack()`, `attack(target)`, `attack(target, damage)` | 같은 기능 이름 통일 || 다중 상속 | 공격 기능 + 회복 기능을 함께 물려받음 | 여러 기능 조합 || static | 전체 챔피언 수 `championCount` | 모든 객체가 공유하는 값 |

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ Class 개념정리	date

발표용 정리

C++의 Class를 게임으로 비유하면, `Champion`은 부모 클래스이고 `Garen`, `Ahri` 같은 챔피언은 자식 클래스이다. 상속은 공통 기능을 물려받는 것이고, 오버라이딩은 같은 `useSkill()`이라도 챔피언마다 다르게 동작하게 만드는 것이다. 가상함수는 부모 타입으로 관리해도 실제 챔피언의 스킬이 실행되게 해준다. 오버로딩은 같은 공격 함수 이름을 상황별로 다르게 쓰는 것이고, static은 전체 챔피언 수처럼 모든 객체가 공유해야 하는 값을 저장할 때 사용한다.

오버로딩과 오버라이딩 비교

오버로딩	같은 이름, 다른 매개변수, 상속 필요 없음
오버라이딩	부모 함수를 자식 클래스에서 재정의, 상속 필요

최종 정리

클래스는 객체를 만들기 위한 설계도이다. 생성자는 객체가 만들어질 때 초기값을 설정하고, 소멸자는 객체가 사라질 때 정리 작업을 한다. 상속은 부모 클래스 기능을 자식 클래스가 물려받는 것이고, 다중 상속은 여러 부모 클래스를 동시에 상속받는 것이다. 오버로딩은 같은 이름의 함수를 여러 형태로 만드는 것이고, 오버라이딩은 부모 함수를 자식 클래스에서 다시 정의하는 것이다. 가상함수는 오버라이딩된 자식 함수가 실제 객체 기준으로 실행되게 하며, static은 클래스 전체가 공유하는 값이나 객체 없이 사용하는 공통 기능을 만들 때 사용한다.

확인 질문

- 클래스와 객체의 차이는 무엇인가?
- 생성자와 소멸자는 각각 언제 실행되는가?
- 상속과 다중 상속의 차이는 무엇인가?
- 오버로딩과 오버라이딩의 차이는 무엇인가?
- 가상함수는 왜 필요한가?
- 순수 가상함수는 어떤 형태로 작성하는가?
- static을 쓰는 이유는 무엇인가?